Chủ đề 8: Animation và cử chỉ nâng cao

Lập trình Đa nền tảng – Dart & Flutter

Trương Nhật Hoàng

Ngày 9 tháng 11 năm 2026

1. Bài toán: thuộc tính UI thay đổi theo thời gian

Khi kích thước, màu, vị trí, trạng thái ẩn/hiện thay đổi, Flutter có 3 cách:

1. **Implicit animation**: chỉ đổi giá trị, Flutter tự chạy chuyển động.
2. **Explicit animation**: tự điều khiển bằng `AnimationController` (chạy, dừng, lặp, đảo chiều).
3. **Gesture**: thao tác người dùng kích hoạt (1) hoặc (2).

Mức độ tự điều khiển tăng dần: $1 \rightarrow 2 \rightarrow 3$.

2. Implicit: AnimatedContainer

- Viết như Container, thêm duration và curve.
- setState đổi giá trị → tự chạy từ giá trị cũ sang mới.

```
GestureDetector(
  onTap: () => setState(() => _expanded = !_expanded),
  child: AnimatedContainer(
    duration: Duration(milliseconds: 300),
    curve: Curves.easeOut,
    height: _expanded ? 160 : 80,
    decoration: BoxDecoration(
      color: _expanded ? Colors.blue.shade100 : Colors.grey.shade200,
      borderRadius: BorderRadius.circular(_expanded ? 24 : 8),
    ),
  ),
)
```

- Đổi giá trị giữa chừng → chạy tiếp từ vị trí hiện tại.

3. Họ widget Animated... có sẵn

Muốn đổi	**Dùng**
Kích thước, màu, bo góc, padding	`AnimatedContainer`
Ẩn/hiện mờ dần	`AnimatedOpacity`
Vị trí căn	`AnimatedAlign`, `AnimatedPositioned`
Thay widget này bằng widget khác	`AnimatedSwitcher`
Giá trị số tùy ý	`TweenAnimationBuilder`

```
AnimatedSwitcher(
  duration: Duration(milliseconds: 200),
  child: Text(_expanded ? 'Chi tiet...' : 'Bam de mo rong',
              key: ValueKey(_expanded)),   // key khac -> widget moi
)
```

4. Khi implicit không đủ

Implicit chỉ chạy **một lần từ A sang B**. Không làm được:

- Lặp vô hạn (icon loading xoay).
- Dừng giữa chừng, đảo chiều theo lệnh.
- Chạy theo vị trí tay kéo.

$\Rightarrow$ Dùng `AnimationController`: một bộ đếm giá trị `value` từ 0.0 đến 1.0, cập nhật mỗi khung hình.

5. AnimationController và quản lý tài nguyên

```
class _S extends State<DemoPage>
    with TickerProviderStateMixin {
  late final AnimationController _spin =
    AnimationController(
      vsync: this,
      duration: Duration(seconds: 1));

  @override
  void dispose() {
    _spin.dispose();   // bat buoc
    super.dispose();
  }
}
```

- vsync: nhịp màn hình gọi cập nhật mỗi khung hình (tương tự ngắt timer).
- Tạo khi State khởi tạo.
- dispose() giải phóng (tương tự destructor C++).
- Quên dispose() → ticker vẫn chạy, rò rỉ tài nguyên.
- Mixin cung cấp vsync:
  1 controller: SingleTickerProviderStateMixin
  nhiều controller: TickerProviderStateMixin

6. Gắn controller vào widget: họ ...Transition

- ScaleTransition, RotationTransition, FadeTransition, SlideTransition.
- Tự vẽ: AnimatedBuilder.
- Lệnh: forward(), reverse(), repeat(), stop(), animateTo(x).

```
// Nut tim nay: 1.0 -> 1.4 -> 1.0
late final _heart = AnimationController(
    vsync: this, lowerBound: 1.0, upperBound: 1.4,
    duration: Duration(milliseconds: 150));

void _like() => _heart.forward(from: 1.0).then((_) => _heart.reverse());

IconButton(
  onPressed: _like,
  icon: ScaleTransition(scale: _heart,
      child: Icon(Icons.favorite, color: Colors.red)),
)

// Icon xoay lien tuc
RotationTransition(turns: _spin, child: Icon(Icons.sync))
onPressed: () => _spin.isAnimating ? _spin.stop() : _spin.repeat()
```

7. Gesture: kích hoạt và điều khiển chuyển động

- `GestureDetector`, `InkWell`: `onTap`, `onDoubleTap`, `onLongPress`, `onHorizontalDragUpdate`, ...
- Có sẵn hiệu ứng: `Dismissible` (vuốt xoá), `Draggable`, `Hero`.

```
GestureDetector(
  // Dang keo: tay dieu khien truc tiep value
  onHorizontalDragUpdate: (d) => _drag.value += d.primaryDelta! / 250,
  // Tha tay: controller tu chay ve mep gan nhat
  onHorizontalDragEnd: (_) => _drag.animateTo(_drag.value > 0.5 ? 1 : 0),
  child: AnimatedBuilder(
    animation: _drag,
    child: Container(width: 60, height: 60, color: Colors.teal),
    builder: (context, child) =>
        Align(alignment: Alignment(_drag.value * 2 - 1, 0), child: child),
  ),
)
```

8. Bảo đảm chuyển động không cản trở thao tác

- **Ngắn**: 150–300 ms cho phản hồi thao tác.
- **Không khoá input**: đang chạy vẫn bấm/kéo được; implicit tự chạy tiếp từ vị trí hiện tại.
- **Build ít**: phần tĩnh truyền qua tham số `child` của `AnimatedBuilder`.
- **Giảm chuyển động**: theo cài đặt hệ thống hoặc công tắc trong app.

```
bool get _reduce =>
    _reduceMotion || MediaQuery.of(context).disableAnimations;

Duration _ms(int ms) =>
    _reduce ? Duration.zero : Duration(milliseconds: ms);

AnimatedContainer(duration: _ms(300), ...)   // chuc nang giu nguyen,
                                              // chi bo chuyen dong
```

9. Demo: một màn hình, 4 nhóm hiệu ứng

Thao tác	Hiệu ứng	Dùng
Bấm card	Mở rộng, đổi màu	`AnimatedContainer`, `AnimatedSwitcher`
Bấm nút tim	Nảy lên rồi về	Controller + `ScaleTransition`
Bấm nút xoay	Xoay liên tục / dừng	`repeat()`, `stop()`
Kéo khối vuông	Theo tay, thả tự về mép	Gesture + `animateTo()`
Vuốt task	Trượt ra và xoá	`Dismissible`
Bật "Giảm chuyển động"	Mọi duration = 0	`_ms()`

3 controller đều được `dispose()` trong `dispose()` của `State`.

10. Tổng kết

- Đổi giá trị A $\rightarrow$ B một lần: **implicit** (`Animated...`).
- Cần lặp, dừng, đảo chiều, theo tay: `AnimationController` + `...Transition`.
- Controller là tài nguyên: tạo trong `State`, `dispose()` khi huỷ.
- Gesture kích hoạt hoặc điều khiển trực tiếp `value`.
- Chuyển động ngắn, không khoá thao tác, tắt được khi cần.